```
  return getPointKey(node.point,
node.level)                                    #2

function getPointKey(point,
level)                                              #3
  j ← level % k
                                                                #4
  return
point[j]
      #5

function compare(point,
node)                                                  #6
  return sign(getPointKey(point, node.level) -
getNodeKey(node))              #7

function splitDistance(point,
node)                                             #8
  return abs(getPointKey(point, node.level) -
getNodeKey(node))               #9
```

#1 Given a tree node, returns the value of the coordinate that needs to be used, given the level at which the node is stored.

#2 In turn, it calls the function extracting this value from the node point.

#3 Given a point (a tuple with `k` values) and an index for the level, returns the tuple entry that should be used in comparisons for nodes at that level.

#4 We assume the method has access to `k`, the dimension of the tree. At level `i`, we need to extract the tuple value at index `i mod k` (`0`-based indexing).

#5 Just return the correct tuple entry.